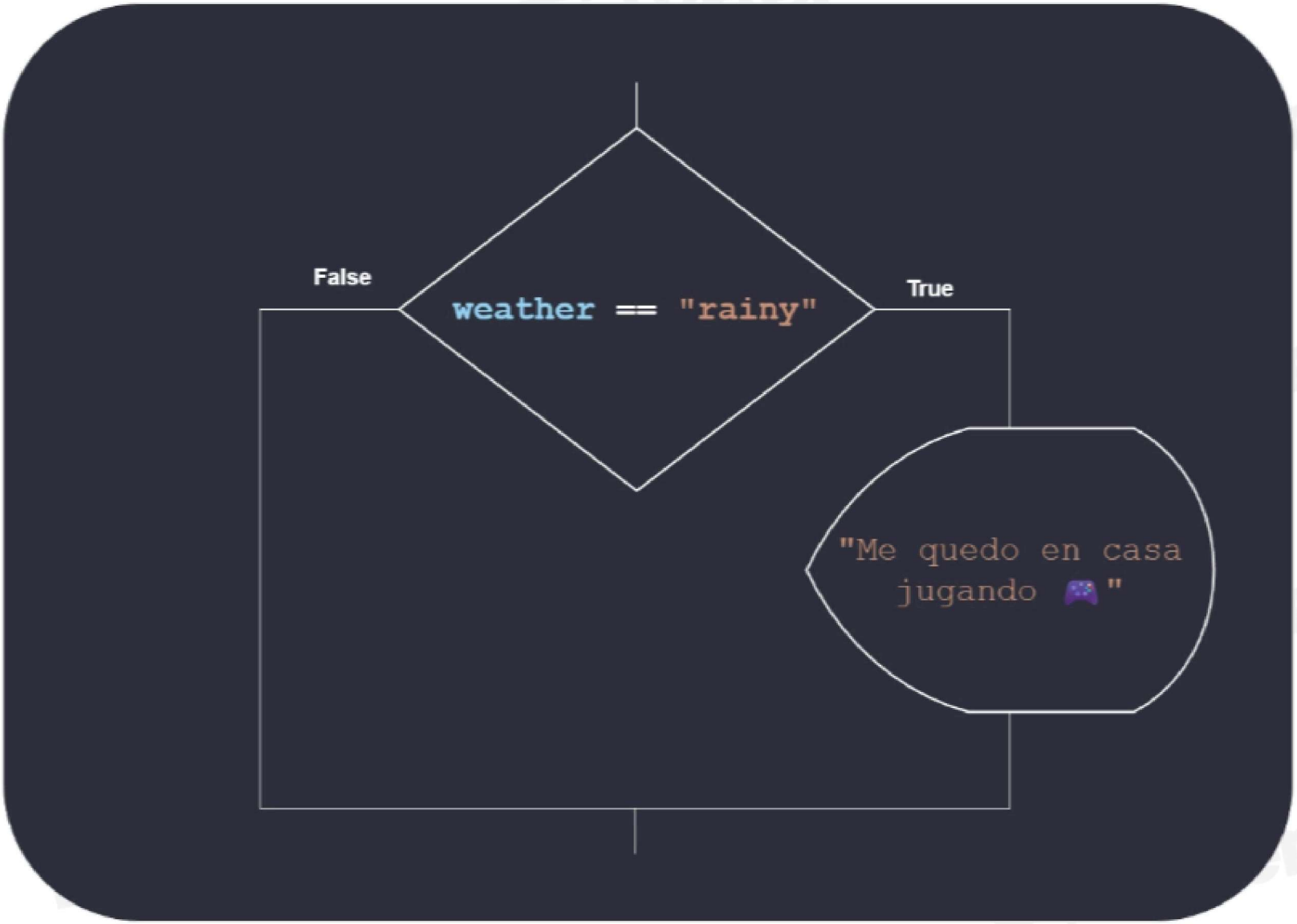
Estructuras Condicionales 🌟

Hasta ahora, nuestros programas han funcionado como un tren que viaja en una sola dirección, comienza en la primera línea de código y avanza hacia abajo, ejecutando una instrucción tras otra en orden, sin desviarse ni detenerse hasta llegar al final. A esto le llamamos **ejecución lineal**.

Las condicionales, al igual que las que vimos en 🔗 Algoritmos y 🔗 Pseudocódigo sirven para hacer que ciertas instrucciones solo se ejecuten si se cumple cierta condición. En esta lección aprenderemos a implementarlas en `Python`.

El bloque `if`

La estructura más básica y fundamental es el `if`. Su funcionamiento es simple: "*Si la condición es verdadera (`True`), ejecuta el siguiente bloque de código. Si es falsa (`False`), ignóralo y continúa*".


False
weather == "rainy"
True
"Me quedo en casa jugando 🎮"

```python
weather = "rainy"

if weather == "rainy":
    # Este código solo se ejecuta si el clima es "rainy"
    print("Me quedo en casa jugando 🎮")
```

Copiar

✓ Me quedo en casa jugando 🎮

En este otro ejemplo, el resultado de la comparación es `False`, por ende, el bloque de código dentro del `if` se omite.

```python
weather = "sunny"

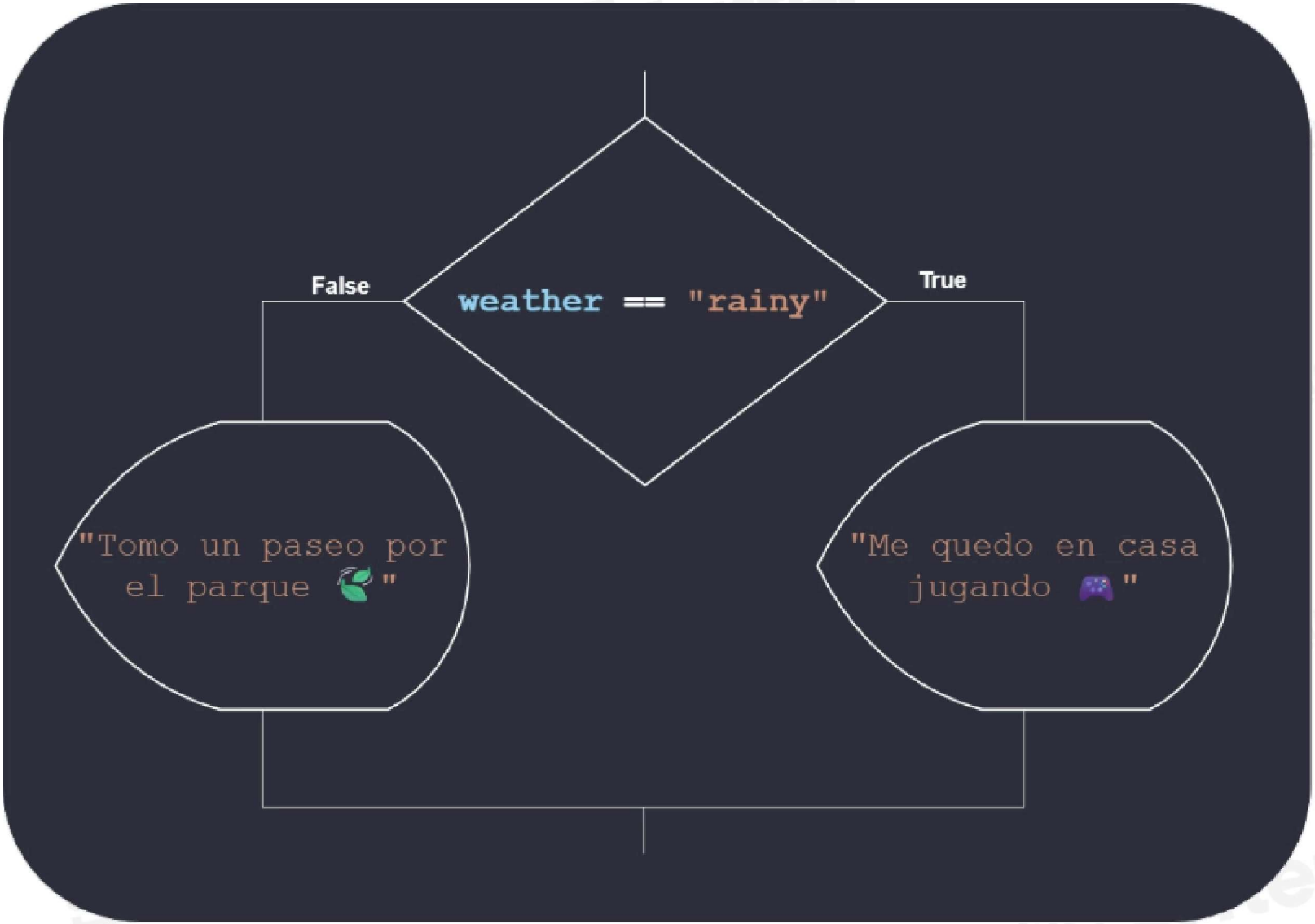
if weather == "rainy":
    print("Me quedo en casa jugando 🎮")
```

Copiar

✓

Alternativas `else`

El bloque `else` lo implementamos cuando queramos que algo suceda si la condición del `if` es `False`. Si la condición se cumple hacemos el plan **A**, si no, hacemos el plan **B**.



Siguiendo con el ejemplo del clima, en este caso se imprime "*Tomo un paseo por el parque* 🌿" porque la condición del `if` es `False`.

```python
weather = "sunny"

if weather == "rainy":
    print("Me quedo en casa jugando 🎮")
else:
    print("Tomo un paseo por el parque 🍃")
```

Copiar

✓ Tomo un paseo por el parque 🍃

Podemos aplicarlo con otros `operadores lógicos` también. Veamos un ejemplo con temperatura:

```python
temperature = 15

if temperature >= 18:
    print("El día está caliente 🌞")
else:
    print("El día está frío ❄️")
```

Copiar

✓ El día está frío ❄️

Múltiples Opciones `elif`

¿Qué pasa si tenemos más de dos opciones? En este caso usamos el operador `elif` que es una abreviatura de "*else if*".

Ya que vamos a tener más de una condición, se evaluará cada una en el orden en que las hayamos colocado. Cuando encuentre una condición verdadera (`True`), ejecutará este bloque y se saltará todo lo demás. Si ninguna de las condiciones es verdadera, ejecutará el bloque `else` .

En este ejemplo, se evalúa primero si el color es verde, luego amarillo, luego rojo y por último, si ninguno coincide, se ejecuta el bloque `else` .

Copiar

```
traffic_light = "red"

if traffic_light == "green":
    print("Avanza 🟢")
elif traffic_light == "yellow":
    print("Precaución 🟡")
elif traffic_light == "red":
    print("Detente 🔴")
else:
    print("Señal desconocida ⚠️")
```

✓ Detente 🔴

Condicionales Anidados 🪆

Conforme nuestros programas crezcan, vamos a encontrarnos con casos más específicos, donde las decisiones dependen de una respuestas previas.

Cuando colocamos un `if` dentro de otro `if` , creamos `condicionales anidados` . Esto nos permite crear una jerarquía de decisiones donde el segundo paso solo ocurre si el primero fue exitoso.

Imaginemos que queremos ir al cine, podemos comprar el boleto, pero si no cumplimos con la edad mínima, no podemos ver la película.

```python
has_ticket = True
age = 16
min_age = 18

if has_ticket:
    print("Tienes boleto, verifiquemos tu edad... 🎟️")

    # Este if vive dentro del anterior
    # Solo se ejecuta si has_ticket es True
    if age >= min_age:
        print("¡Bienvenido a la función! 🍿")
    else:
        print("Lo sentimos, eres menor de edad para esta película 🔞")

else:
    print("Por favor compra un boleto en taquilla 🎫")
```

Copiar

✓ Lo sentimos, eres menor de edad para esta película 🔞

Match Case 🔀

En algunos casos, donde tengamos que validar muchos valores sobre una sola variable, utilizar las estructuras `if` y `elif` puede volverse tedioso y confuso. Para simplificar esto, `Python` introdujo en su versión 3.10 la estructura `match`, conocida en otros lenguajes como *`Switch Case`*.

Esta estructura toma un valor y lo compara con diferentes casos. Si coincide, ejecuta el código de ese bloque.

Veamos un ejemplo con un menú de opciones:

Copiar

```python
# Solicitamos la opción al usuario
option = int(input("Selecciona una opción (1-3): "))

# Siempre validamos el valor de la variable option
match option:
    case 1:
        print("Ver perfil 👤")
    case 2:
        print("Ir a Configuración ⚙️")
    case 3:
        print("Cerrar sesión 👋")
    case _:
        print("Opción no válida 🚫")
```

💡 Nota el `case _:` al final. Este actúa como un comodín (*similar al* `else`), ejecutándose si ninguno de los casos anteriores coincide.

Si quisiéramos replicar este comportamiento usando estructuras `if` deberíamos hacerlo como se muestra a continuación.

Notemos cómo siempre se hace la condición usando la variable `option`, por lo que para evitar repetir tantas veces el mismo código, `match` es una gran alternativa.

```python
# Solicitamos la opción al usuario
option = int(input("Selecciona una opción (1-3): "))

# Siempre validamos el valor de la variable option
if (option == 1):
    print("Ver perfil 👤")
elif (option == 2):
    print("Ir a Configuración ⚙️")
elif (option == 3):
    print("Cerrar sesión 👋")
else:
    print("Opción no válida 🚫")
```

Copiar

La Indentación ➡️

¿Notaste que en todos los ejemplos el código dentro de los condicionales está un espacio a la derecha? Esta indentación en `Python` representa un bloque de código y para crearla usamos la tecla `Tab` de nuestro teclado.

Todo lo que esté indentado debajo del `if` es parte de ese condicional.

Veamos un ejemplo sencillo, donde la impresión del texto "*Estoy dentro del bloque*" depende del resultado de la condición. En cambio el texto "*Estoy fuera del bloque*" sin importar el caso, se va a ejecutar.

```python
number = 25
if number > 50:
    print("Estoy dentro del bloque")

print("Estoy fuera del bloque")
```

Copiar

✓ Estoy fuera del bloque

Y aquí un ejemplo más complejo con múltiples niveles anidados:

Copiar

```python
country = "Costa Rica"
state = "Guanacaste"
city = "Liberia"

if country == "Costa Rica":
    print("Bienvenido a Costa Rica CR")
    if state == "Guanacaste":
        print("Bienvenido a Guanacaste 🏖️")
        if city == "Liberia":
            print("Bienvenido a Liberia ✈️")
        else:
            print("No estás en Liberia")
    else:
        print("No estás en Guanacaste")
else:
    print("No estás en Costa Rica")
```

✓ Welcome to Costa Rica

Welcome to Guanacaste

Welcome to Liberia

💡 **Nota Importante**: Si olvidas la indentación, `Python` te dará un error o el programa no funcionará correctamente. ¡Las indentaciones son parte de la sintaxis!

Buenas prácticas ✔️

Antes de concluir con la lección, una pequeña nota adicional sobre buenas prácticas. Recordemos que para que las instrucciones de una condición se ejecuten, la misma debe evaluarse como verdadera (`True`).

Esto significa que hacer comparaciones explícitas con `True` o `False` es redundante y está mal visto. Vamos a simplificarlo:

Cuando queremos validar si es `True` :

Copiar

```
is_raining = True

# Redundante (Evita esto)
if is_raining == True:
    print("Está lloviendo")

# Correcto
# "Si está lloviendo..."
if is_raining:
    print("Está lloviendo")
```

Cuando queremos validar si es `False` :

Para esto nos apoyamos en el operador `not` .

Copiar

```python
is_sunny = False

# Redundante
if is_sunny == False:
    print("No hay sol")

# Correcto
# "Si NO hay sol..."
if not is_sunny:
    print("No hay sol")
```
